RICHFIELD
richfield.ac.za

Name and Surname:

_____

Student ITS No: _____

Qualification: _____ **Year of Study:** _____ **Semester:** _____

Assignment due date: _____ **Date submitted:** _____

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents

Introduction

Main Body (with relevant subheadings)

Conclusion

References

Submit the assignment in PDF format on Moodle.

Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: **Date:**

Contents

Introduction 3

System Design 4

Screen flow 4

Data Model 5

The screenshots 6

Challenges and solutions 12

Conclusions 12

References 14

Introduction

Most of us toss out more food than we'd care to admit. That bag of spinach gets forgotten in the back of the fridge and turns to mush, a chunk of cheese hardens up, and those tomatoes we grabbed with good intentions end up in the trash. Here's the thing though: usually it's not that we're unwilling to cook, it's that we're stumped by the random assortment of ingredients sitting in our kitchen, so we order takeout instead and watch those leftovers pile up.

Smart Pantry Manager is the solution to that exact problem. It's a native Android app built with Java that tracks the ingredients you actually have on hand and recommends recipes you can prepare using only those ingredients. There's no shopping list feature, no "grab two more items and you're set" suggestions, and intentionally no store locator. When the app recommends a recipe, you can make it today with what's already in your kitchen.

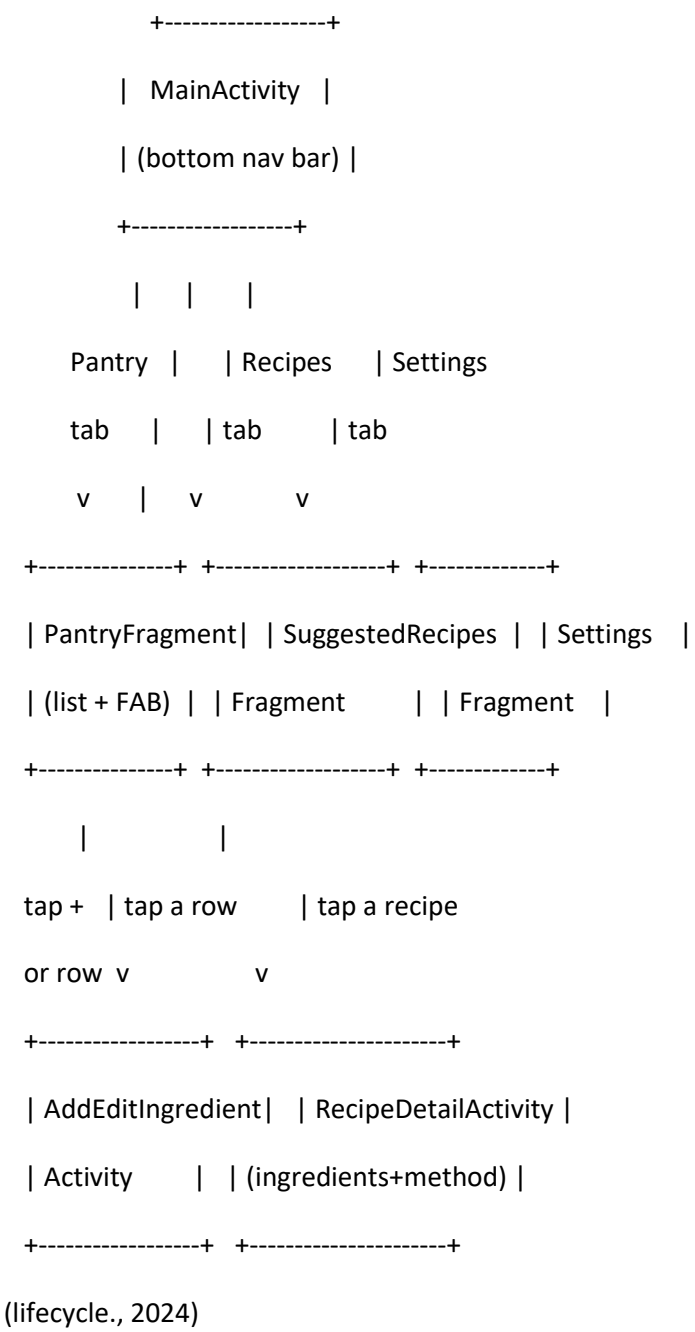
Its designed for everyday home cooks, students, and budget-conscious folks who want to cut down on waste and spending. It's deliberately lightweight and works completely offline. All your data stays on your device, keeping everything private and letting you use it anywhere, even without internet.

In this report, I walk through how the app works and demonstrate each screen in action. Then I highlight the code sections I think are most important and discuss the bugs that gave me trouble, since several of them taught me valuable lessons that the smooth-running parts never could. (Developers, 2024)

System Design

Screen flow

The app is built around a single host Activity (MainActivity) that holds a bottom navigation bar. Three main screens live as Fragments inside that host, and two more open as their own Activities when you tap through to them. That gives five screens in total, connected as follows:



(lifecycle., 2024)

The bottom navigation bar lets you switch between the three primary tabs. Reaching the Add/Edit screen and the Recipe Detail screen works in a different way: they both launch as independent Activities, triggered by an explicit Intent, and you pass the information along as an Intent extra (either the row id or the recipe id). (Anon., 2024)

Data Model

I went with SQLite as my storage solution, leveraging SQLiteOpenHelper to manage the database. My data structure consists of three interconnected tables: one tracking what users have in their possession, another storing recipes, and a third listing the ingredients required for each recipe. The connection between recipes and their ingredients follows a one-to-many pattern, meaning each recipe can call for multiple ingredients.

Table	Column	Type	Notes
pantry_items	_id	INTEGER PK	auto-increment
pantry_items	name	TEXT	display name, e.g. "Tomatoes"
pantry_items	name_norm	TEXT	normalised name used for matching
pantry_items	quantity	REAL	how much the user has
pantry_items	unit	TEXT	g, kg, ml, l, pcs, etc.
pantry_items	expiry_date	TEXT	optional, yyyy-MM-dd
recipes	_id	INTEGER PK	auto-increment
recipes	name	TEXT	recipe title
recipes	steps	TEXT	preparation method
recipe_ingredients	_id	INTEGER PK	auto-increment

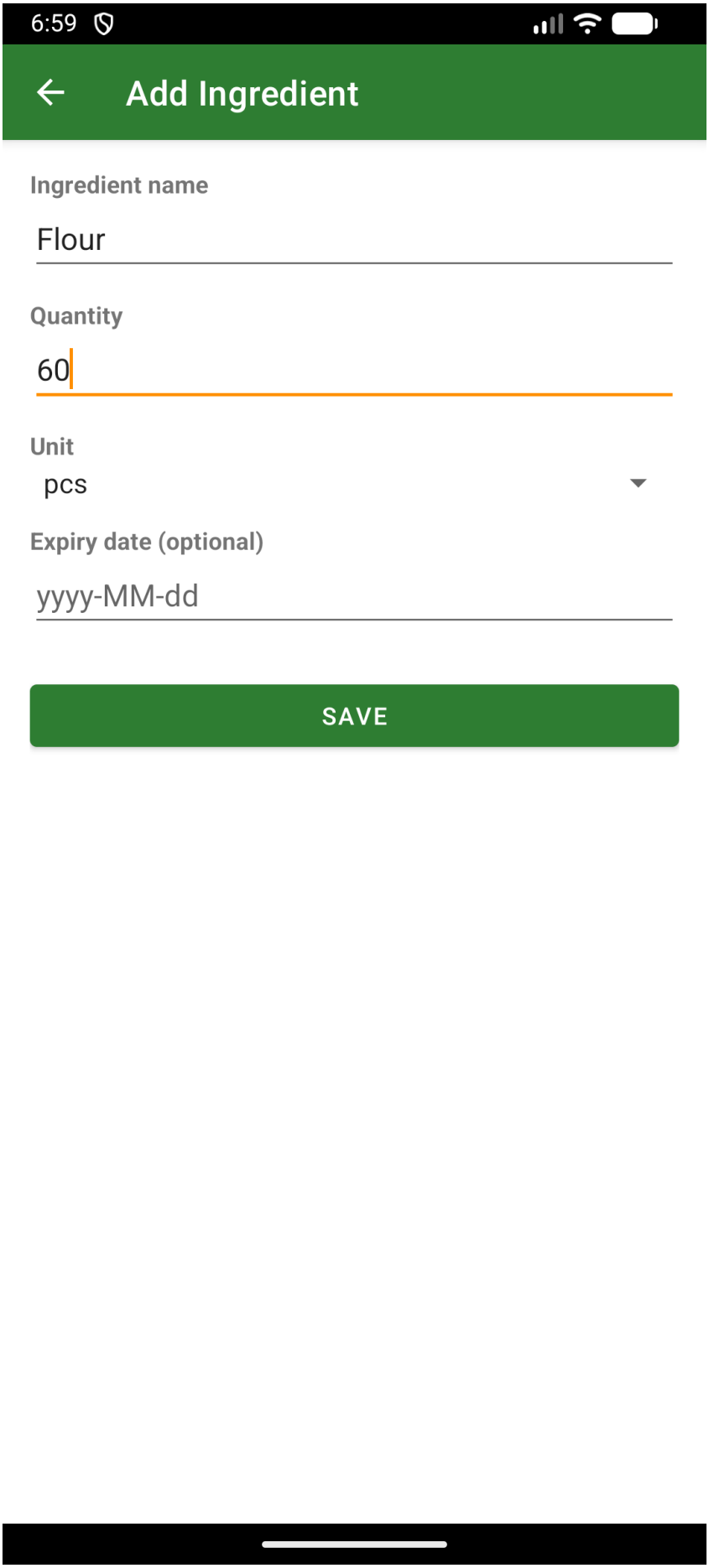
Table	Column	Type	Notes
recipe_ingredients	recipe_id	INTEGER FK	references recipes(_id)
recipe_ingredients	name	TEXT	required ingredient name
recipe_ingredients	quantity	REAL	amount needed
recipe_ingredients	unit	TEXT	unit of the amount

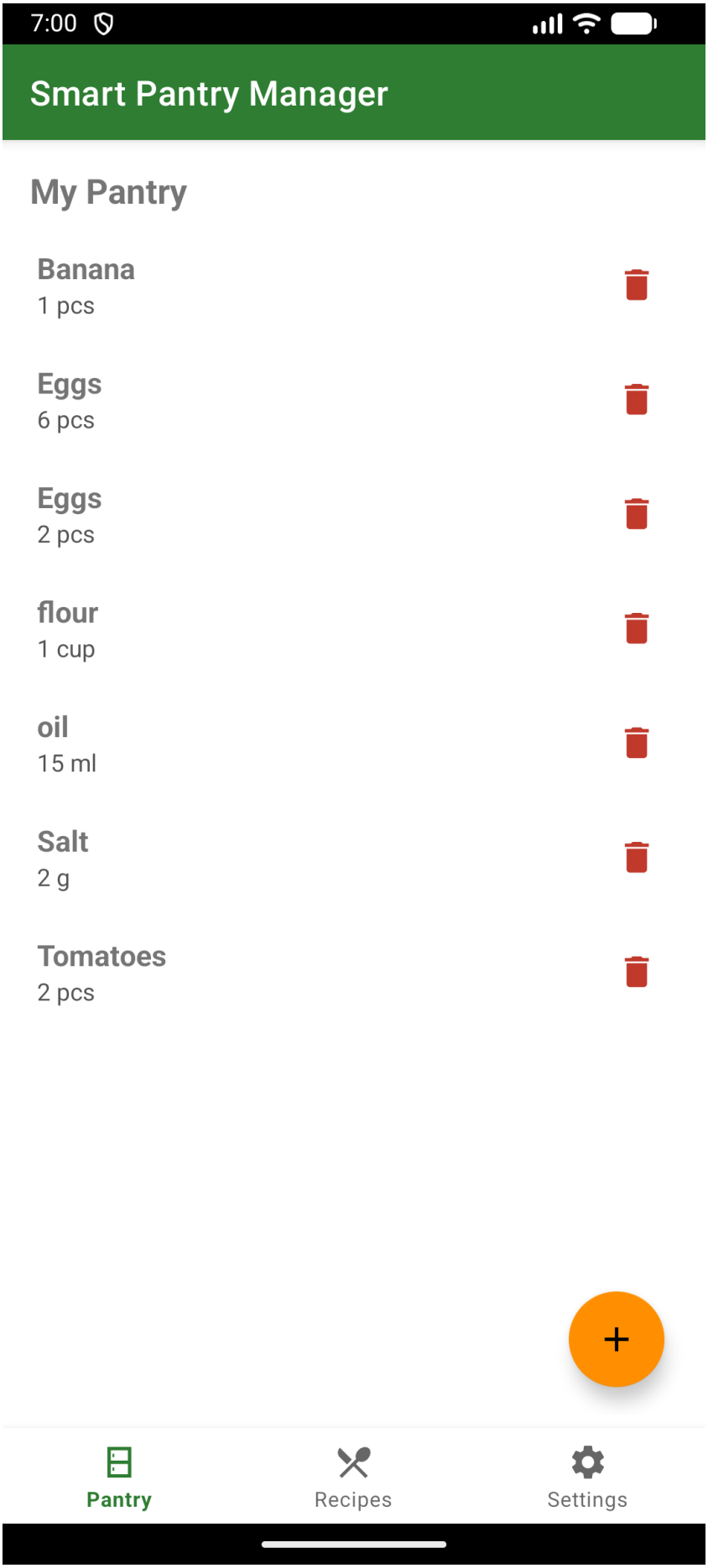
Here's how it works in simple terms: each recipe can have multiple ingredients listed in the recipe_ingredients table, and they're connected through the recipe_id. The pantry_items table stands apart from this connection instead of using a SQL join, the matching logic compares them at runtime. This approach makes sense because the comparison requires name normalization and unit conversion, which are more straightforward to handle in Java code rather than in SQL. (Anon., 2026)

The screenshots

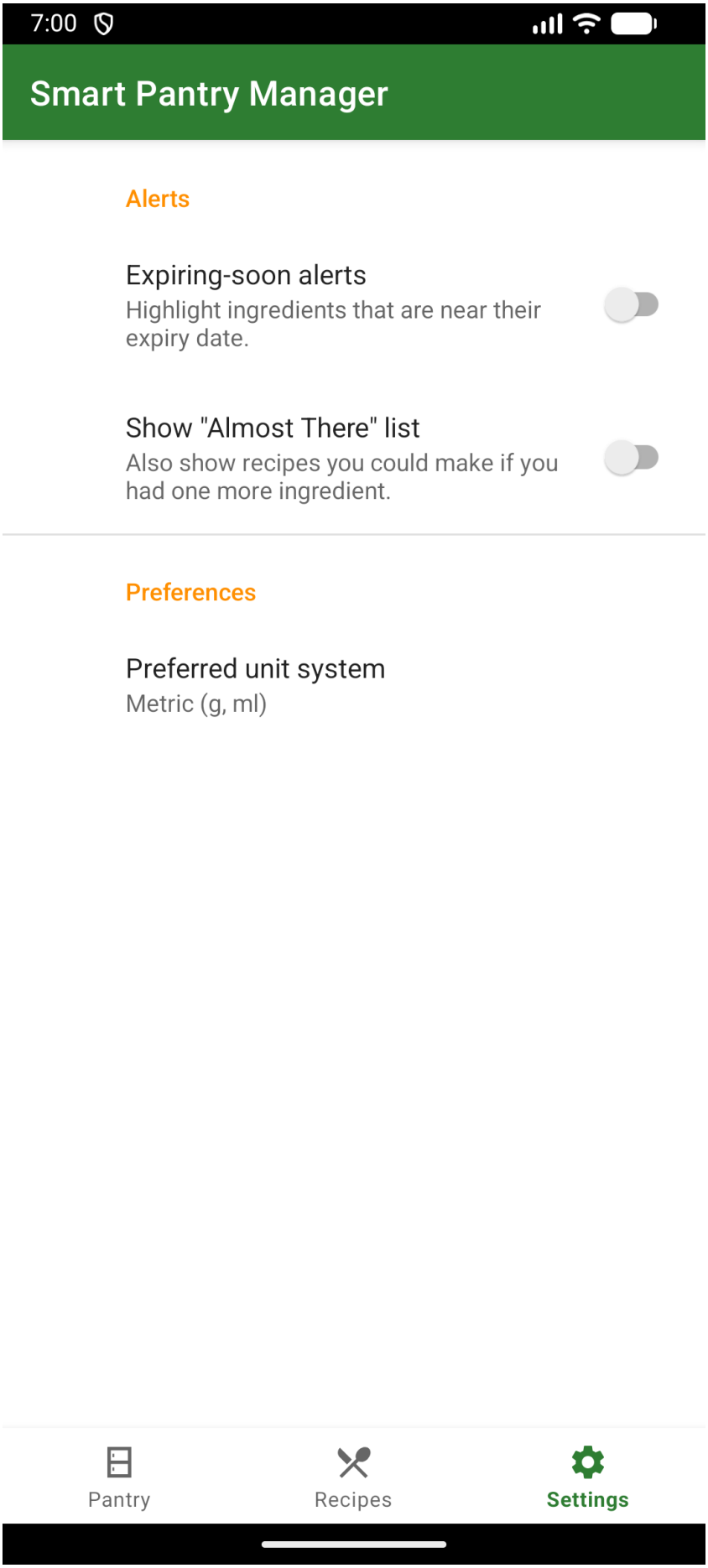
The screenshots were taken from the app directly I took the pictures but then the app started glitched so the I couldn't take a picture of the rest***

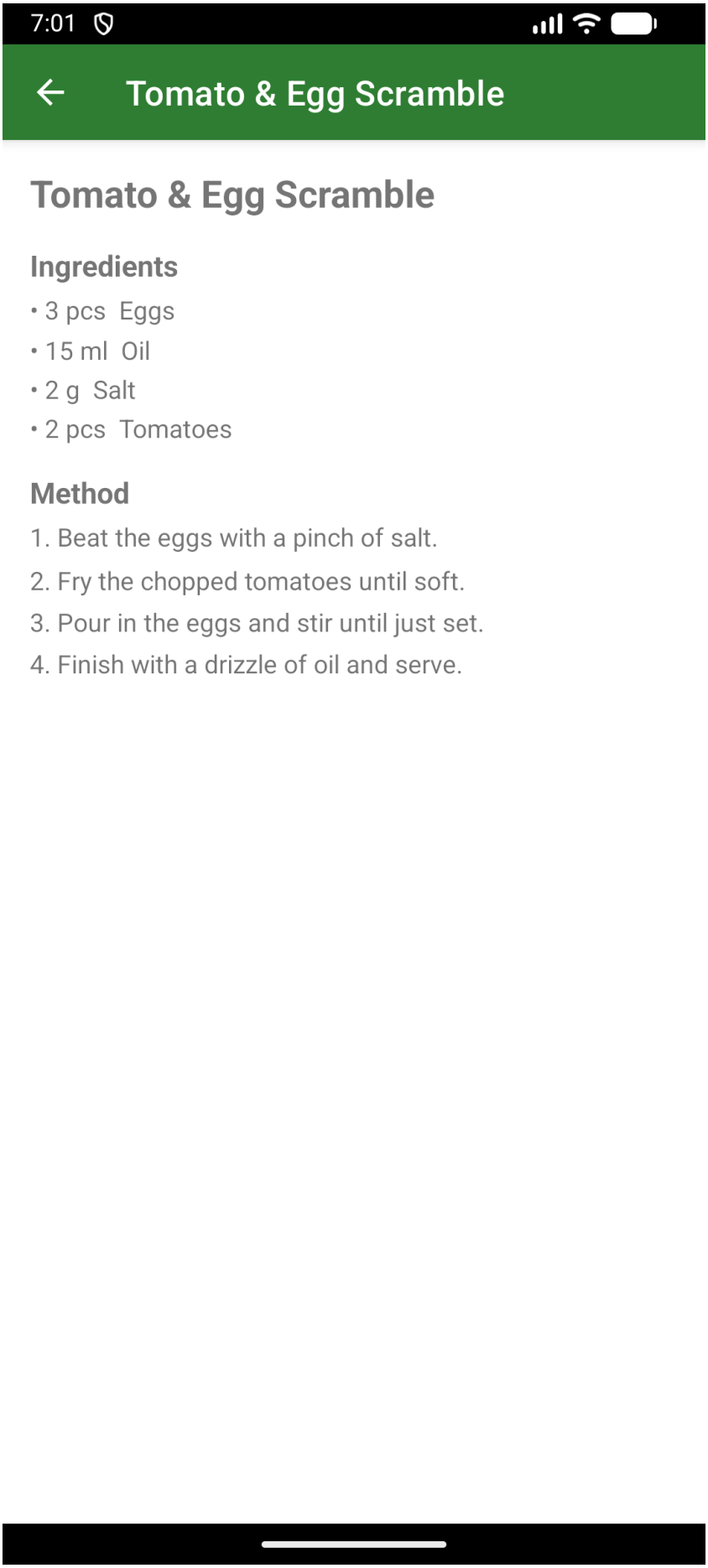
1) The first picture is an add ingredient form that's filled in
   Creating a new pantry item. The form captures name, quantity, unit and an optional expiry date.
2) Is the Pantry list with a few listed ingredients
   The Pantry screen showing the current ingredients in a RecyclerView, with the add button in the bottom corner.
3) This is showing a suggested recipe
4) The last two is the settings and more information on the suggested recipe


6:59
Add Ingredient
Ingredient name
Flour
Quantity
60
Unit
pcs
Expiry date (optional)
yyyy-MM-dd
SAVE


7:00
Smart Pantry Manager
My Pantry
Banana
1 pcs
Eggs
6 pcs
Eggs
2 pcs
flour
1 cup
oil
15 ml
Salt
2 g
Tomatoes
2 pcs
Pantry
Recipes
Settings


7:00
Smart Pantry Manager
Suggested Recipes
Tomato & Egg Scramble
4 ingredients
Pantry
Recipes
Settings


7:00
Smart Pantry Manager
Alerts
Expiring-soon alerts
Highlight ingredients that are near their expiry date.
Show "Almost There" list
Also show recipes you could make if you had one more ingredient.
Preferences
Preferred unit system
Metric (g, ml)
Pantry
Recipes
Settings


7:01
Tomato & Egg Scramble
Tomato & Egg Scramble
Ingredients
• 3 pcs Eggs
• 15 ml Oil
• 2 g Salt
• 2 pcs Tomatoes
Method
1. Beat the eggs with a pinch of salt.
2. Fry the chopped tomatoes until soft.
3. Pour in the eggs and stir until just set.
4. Finish with a drizzle of oil and serve.

Challenges and solutions

Simple naming inconsistencies were causing matches to fail. Initially, I used a straightforward .equals() check to compare ingredient names, which worked fine until I encountered a recipe listing "Tomatoes" while my pantry had "tomato." The recipe disappeared without any error message, and I mistakenly thought the matching logic was flawed. It turned out the problem was just the string comparison. I solved this by creating an IngredientNormalizer class that converts both sides to lowercase and converts plurals to singular forms before any comparison happens. I kept the solution intentionally straightforward rather than using a complex language library, since the requirements emphasized reliability over sophisticated language handling.

The recipe suggestions weren't updating after adding ingredients. When I'd switch back to the Recipes tab following an ingredient addition, the list remained unchanged, making me suspect the save operation had failed. The save was actually working fine the Fragment simply wasn't recalculating. I fixed this by moving the recalculation logic into onResume(), ensuring the strict-matching process runs whenever the screen comes back into view. This single adjustment made the add-remove-reappear workflow function exactly as users would expect.

Recipes from the seed data were multiplying unexpectedly. I was initially seeding recipes from the Activity during every app launch, which meant after three sessions I'd have three identical copies of each recipe. I resolved this by moving the seeding operation into the SQLiteOpenHelper's onCreate() method, which Android executes only once when the database file is initially created. Now recipes load precisely once and continue to persist through app restarts.

Conclusions

I picked up way more about the Android lifecycle by actually building this than I ever did sitting through lectures, mainly because I kept messing it up. The onResume() refresh bug and the duplicate-seeding bug turned out to be the same mistake happening twice: I kept assuming a method would run when I needed it to, rather than when Android actually triggers it. Having to work through those fixes pushed me to stop making assumptions and genuinely understand how the lifecycle operates. (Settings., 2026)

The matching logic is what I'm most proud of. I kept it isolated in its own small class without any UI code tangled in, which meant I could test the strict rule and the plural handling using plain Junit no emulator required. Being able to run those tests in just a couple of seconds is what gave me confidence that the rule actually works the way it's supposed to.

If I had more time, I'd add a feature letting users build their own recipes instead of just using the pre-loaded ones, make the expiry handling more robust so items actually show as expiring soon, and dedicate more effort to the visual design. I'd also explore using a SQL query for the matching when dealing with larger recipe collections, since my current method pulls everything into memory, which works fine for 18 recipes but wouldn't hold up well with thousands.

References

Anon., 2024. *Create dynamic lists with RecyclerView..* [Online] Available at: https://developer.android.com/develop/ui/views/layout/recyclerview [Accessed 23 October 2026].

Anon., 2026. *Sending the user to another app with intents.* [Online] Available at: https://developer.android.com/training/basics/intents/sending [Accessed 23 October 2026].

Developers, A., 2024. *Android Developers. (2024) Save data using SQLite.* [Online] Available at: https://developer.android.com/training/data-storage/sqlite [Accessed 23 October 2026].

lifecycle., T. a., 2024. *Android Developers. (2024) The activity lifecycle..* [Online] Available at: https://developer.android.com/guide/components/activities/activity-lifecycle [Accessed 23 October 2026].

Settings., A. D. (., 2026. [Online] Available at: https://developer.android.com/develop/ui/views/components/settings [Accessed 26 October 2026].